Funciones en Python

Creando bloques reutilizables de código

Módulo IV: Estructura y Modularidad

Objetivos de la Clase



Concepto

Comprender qué es una función y por qué es vital en programación.



Sintaxis

Aprender a definir funciones con parámetros y valores de retorno.



Práctica

Implementar un proyecto real de Inventario por Consola.

¿Qué es una función?



La Analogía de la Licuadora

Una función es como una licuadora:

- **Entrada:** Frutas (datos).
- **Proceso:** Licuado (código).
- **Salida:** Jugo (resultado).

La usas cuando la necesitas, cuantas veces quieras, sin tener que construirla cada vez.

¿Por qué usar funciones?

Problema: Repetición

Escribir el mismo código una y otra vez hace que tu programa sea difícil de mantener y propenso a errores.

```
print("Bienvenido al Sistema")
print("Bienvenido al Sistema")
```

Solución: Reutilización

Agrupas el código bajo un nombre y lo llamas cuando lo necesites. ¡Principio DRY (Don't Repeat Yourself)!

```
def saludar():
    print("Bienvenido al Sistema")
```

Sintaxis Básica

```
def nombre_de_la_funcion():
    # Bloque de código indentado
    print("Tarea realizada")
```

- ✓ `def` indica que estamos definiendo una función.
- ✓ El nombre debe ser descriptivo y seguir las reglas de variables.
- ✓ Los **paréntesis** y los **dos puntos** son obligatorios.

Invocación o Llamada

Definir la función **no la ejecuta**. Para que el código dentro de ella funcione, debemos "llamarla" por su nombre.

```
def saludar():
    print("¡Hola Mundo!")

# Aquí ejecutamos la función
saludar()
```



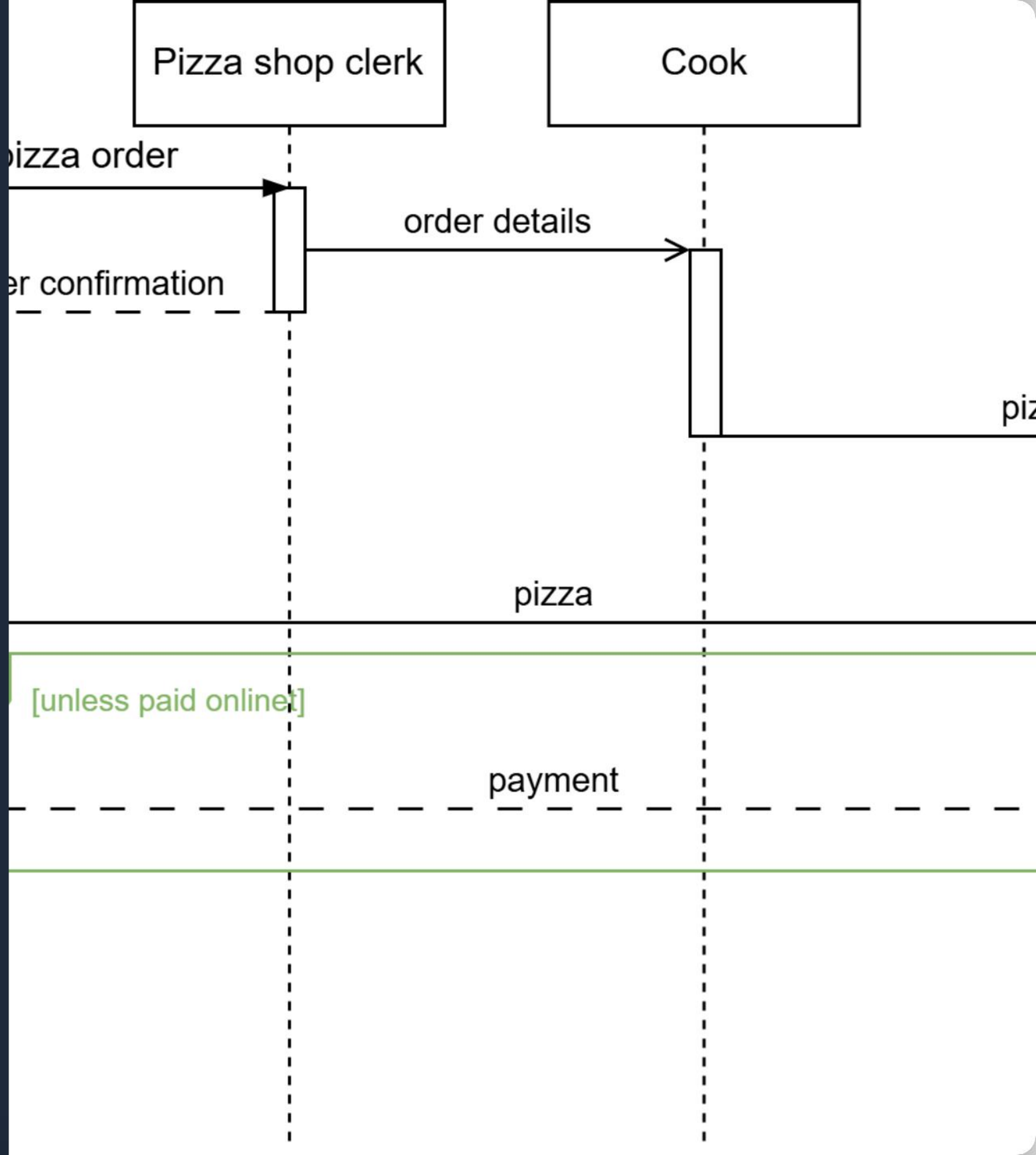
Llamada directa

Simplemente escribes el nombre seguido de paréntesis.

Flujo de Ejecución

Cuando Python encuentra una llamada a una función:

- 1 El programa **salta** a la definición de la función.
- 2 Ejecuta todo el código interno.
- 3 **Regresa** a la línea siguiente donde fue llamada.



Funciones con Parámetros

Personalización

Los parámetros permiten que la función reciba información externa para trabajar.

```
def saludar(nombre):
    print("Hola", nombre)
```

Uso dinámico

Puedes pasar diferentes valores en cada llamada:

```
saludar("Juan")
saludar("María")
```

Varios Parámetros

n

Argumentos ilimitados

Una función puede recibir múltiples datos separados por comas. El orden es fundamental.

```
def sumar(a, b):
    print(a + b)

sumar(10, 5) # Resultado: 15
```

Comunicación de Salida

> *El valor de una función no es lo que muestra, sino lo que nos devuelve para seguir trabajando.*

- La importancia del `return`

La Sentencia return

Permite que la función **entregue** un resultado al programa principal, en lugar de solo imprimirlo.

```
def multiplicar(x, y):
    return x * y
```



Punto final

Cuando Python encuentra un `return`, la función termina inmediatamente.

Capturando el Resultado

Como la función devuelve algo, podemos guardarlo en una variable:

```
resultado = multiplicar(4, 5)
print("El área es:", resultado)
```

Resultado = a × b

Print() vs Return

Acción	Print()	Return
Propósito	Mostrar información al humano.	Entregar datos a otra parte del código.
Reutilización	Dificil (el dato se queda en consola).	Alta (puedes operar con el dato).
Analogía	Gritar el resultado.	Entregar un papel con el resultado.

Ejemplo: Calculadora de IVA

```python
def calcular_iva(precio):
    total = precio * 1.21
    return total

p = 1000
final = calcular_iva(p)
print(f"Total: {final}")
```

Lógica limpia

La función se encarga de la matemática, el programa principal se encarga de mostrarla.

Proyecto Integrador

Mini-Inventario por Consola

Estructura del Inventario



Utilizaremos una **Lista de Diccionarios** para modelar nuestra base de datos en memoria.

```
productos = [
  {"nombre": "Mouse", "precio": 15000},
  {"nombre": "Teclado", "precio": 25000}
]
```

División de Responsabilidades



Agregar

Recibe datos y hace `.append()`.



Listar

Recorre la lista con un `for` y muestra cada item.



Buscar

Compara el nombre ingresado con el diccionario.

Código: Agregar Producto

```
def agregar_producto(nombre, precio):
    nuevo = {"nombre": nombre, "precio": precio}
    inventario.append(nuevo)
    print("¡Producto agregado con éxito!")
```

Esta función encapsula toda la lógica de creación. No importa dónde la llames, siempre hará lo mismo.

El Bucle del Menú

```
while True:
    print("1. Agregar | 2. Ver | 3. Salir")
    opcion = input("Elige: ")
    if opcion == "1": agregar_producto(...)
    elif opcion == "3": break
```

Integramos todo: Ciclos, Condicionales y Funciones.

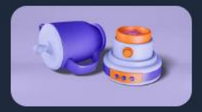
¡Felicidades!

- ✓ Modularidad con funciones.
- ✓ Uso de parámetros y `return`.
- ✓ Integración de datos complejos.

¿Preguntas sobre el Proyecto Integrador?

Próxima clase: Programación Orientada a Objetos

Image Sources



https://img.magnific.com/free-psd/juicer-blender-kitchen-appliance-icon-isolated-3d-render-illustration_439185-12436.jpg

Source: www.magnific.com



https://static.tildacdn.com/tild3335-3939-4130-b262-306531633662/__2drawio_5.png

Source: shesterov.by



https://png.pngtree.com/thumb_back/fw800/background/20260409/pngtree-modern-powerful-gaming-pc-setup-with-ultrawide-monitor-mechanical-keyboard-and-image_21693609.webp

Source: pngtree.com



https://w0.peakpx.com/wallpaper/41/600/HD-wallpaper-connectivity-digital-blue-abstract-background-android-technology.jpg

Source: www.peakpx.com